

Banker's Algorithm – Deadlock Avoidance and Detection

Visualizing Safe Resource Allocation in Operating Systems

Operating Systems | Usman Institute of Technology

M.Ahsan (24FA-059-CS)	M.Ahmad (24FA-050-CS)	Umais Jan (24FA-009-CS)	Umar Sani (24FA-038-CS)
--------------------------	--------------------------	----------------------------	----------------------------

Structural Blueprint of the Presentation

01

Project Overview & Motivation



02

OS Concepts: The Deadlock Dilemma



03

Banker's Algorithm Logic & Trace

	0	1	... 0
	1	0	... 1
	0	1	... 0
	1	0	... 1

04

System Design & Architecture



05

Simulator Demonstration



06

Key Findings & Limitations



Visualizing Deadlock Avoidance

The Problem

Deadlocks permanently freeze operating systems by locking up competing resource requests, forcing highly disruptive system reboots.

The Solution Strategy

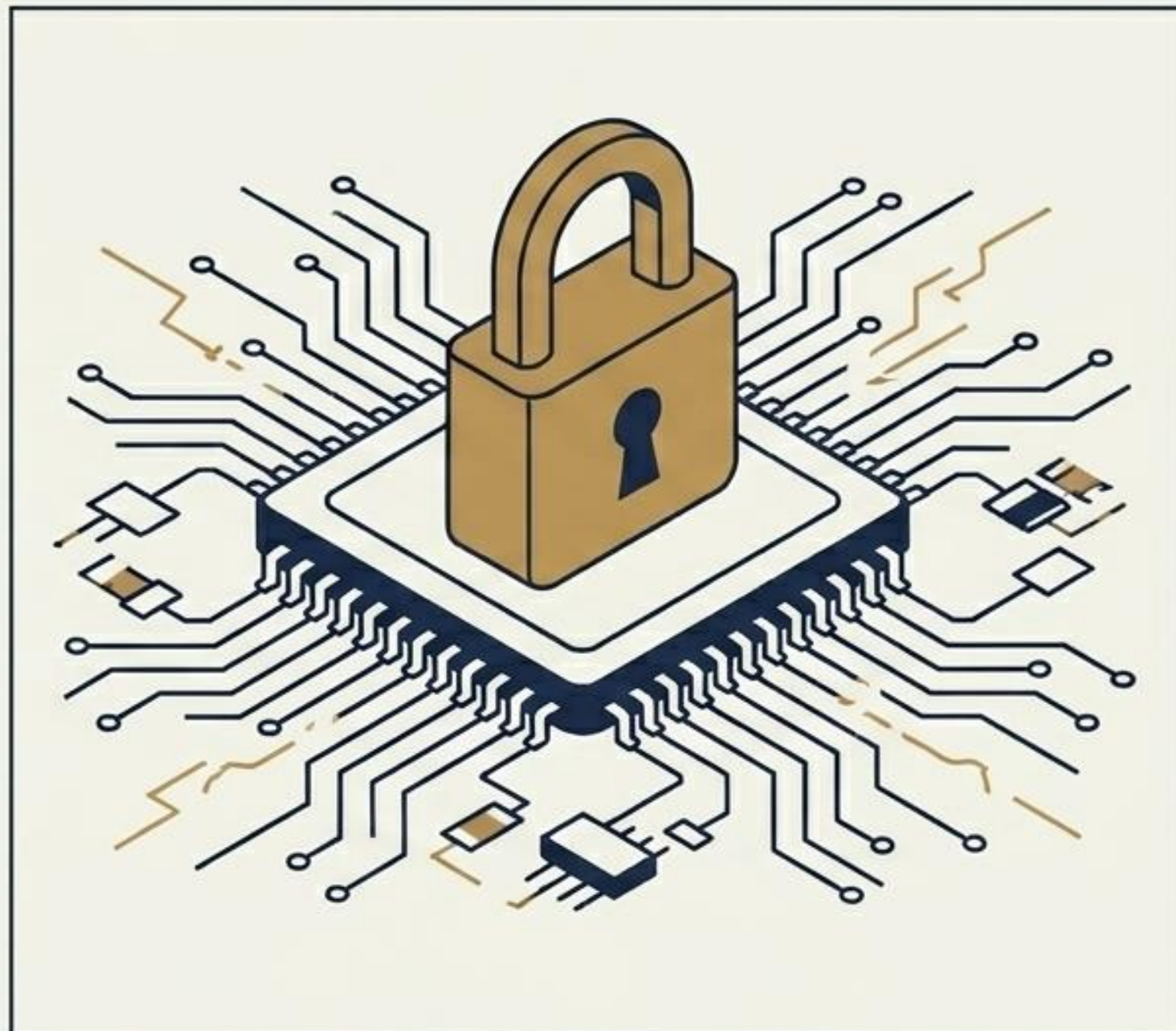
Proactive deadlock avoidance is mission-critical for maintaining unbroken system stability.

The Project

A custom-built Python Banker's Algorithm simulator powered by an interactive Tkinter GUI.

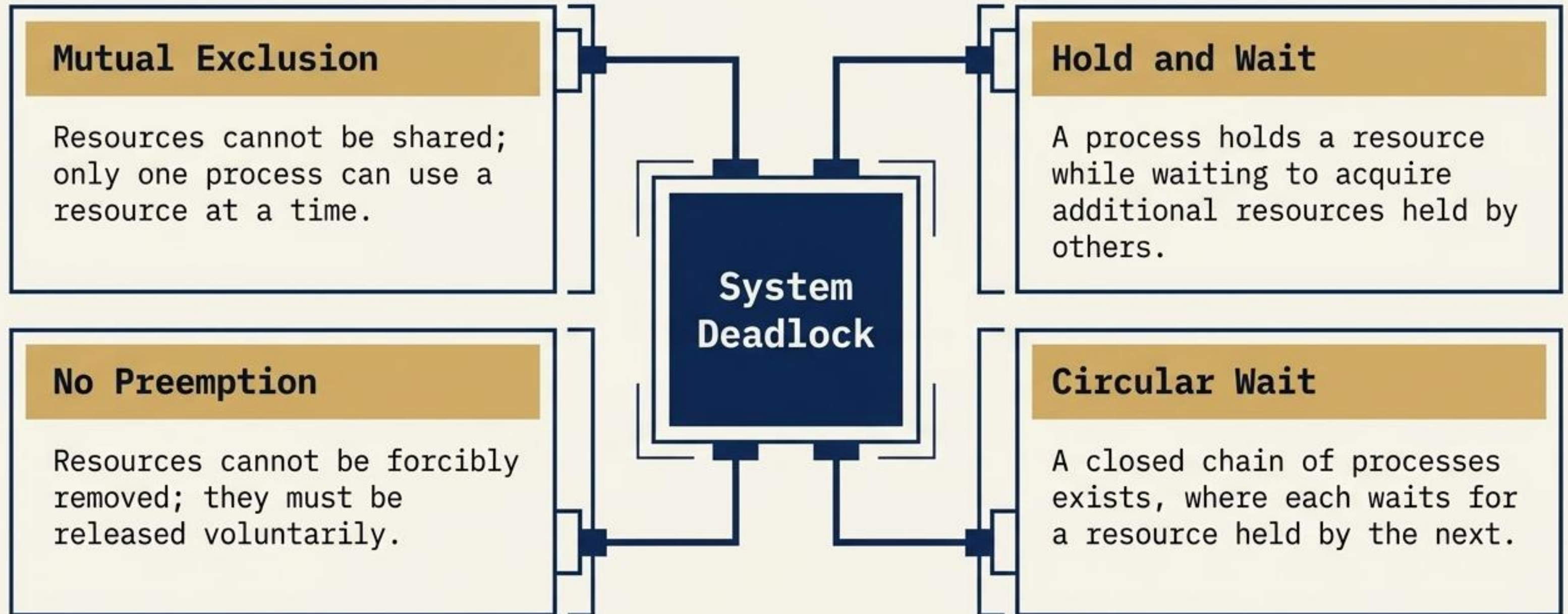
Our Motivation

To transform a highly abstract, classic OS algorithm into a dynamic, visual learning tool.



The Deadlock Dilemma: Four Necessary Conditions

A deadlock is a system state where processes are indefinitely blocked, each waiting for a resource currently held by another.



Handling Strategies and Resource Mapping

Resource Mapping

Process: The active software task requesting access.

Resource Type: The category of hardware/software (e.g., CPU, Printers).

Instance: The specific, available unit of that resource category.

Prevention

Structurally denies at least one of the four deadlock conditions.

High constraint, low resource utilization.

Avoidance (Our Focus)

Dynamically analyzes incoming requests to ensure the system never enters an unsafe state.

Powered by Banker's Algorithm.

Detection & Recovery

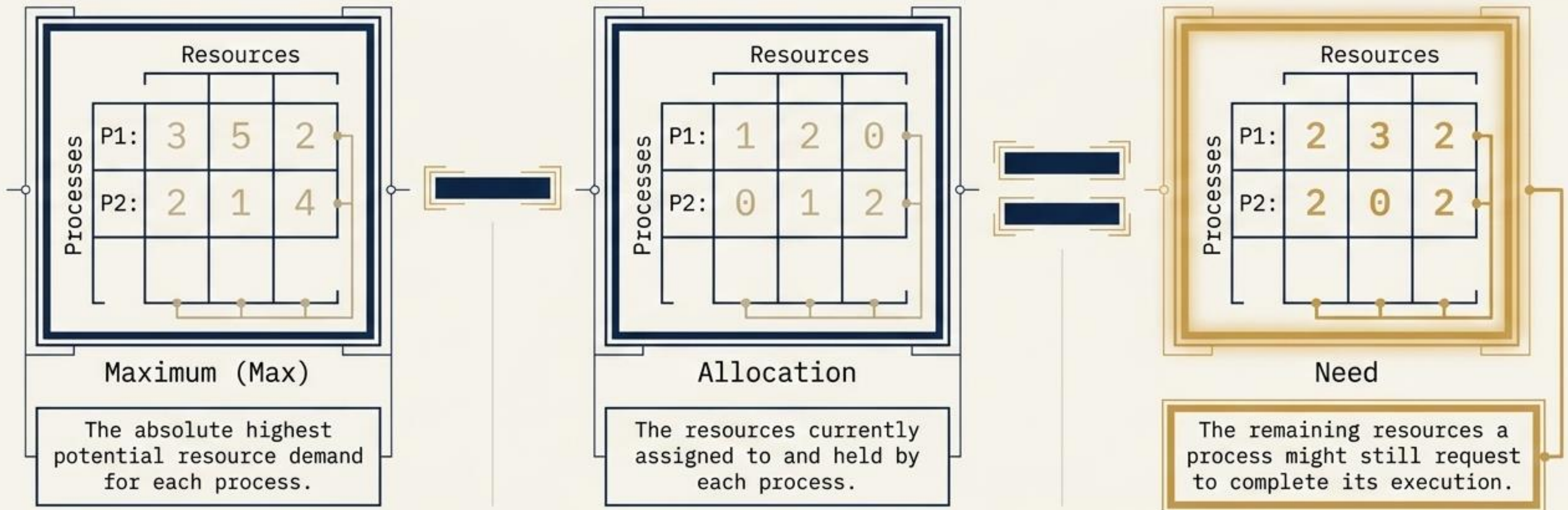
Allows deadlocks to happen, periodically scanning the system to abort processes and recover.

High overhead, highly disruptive.

The Core Data Structures

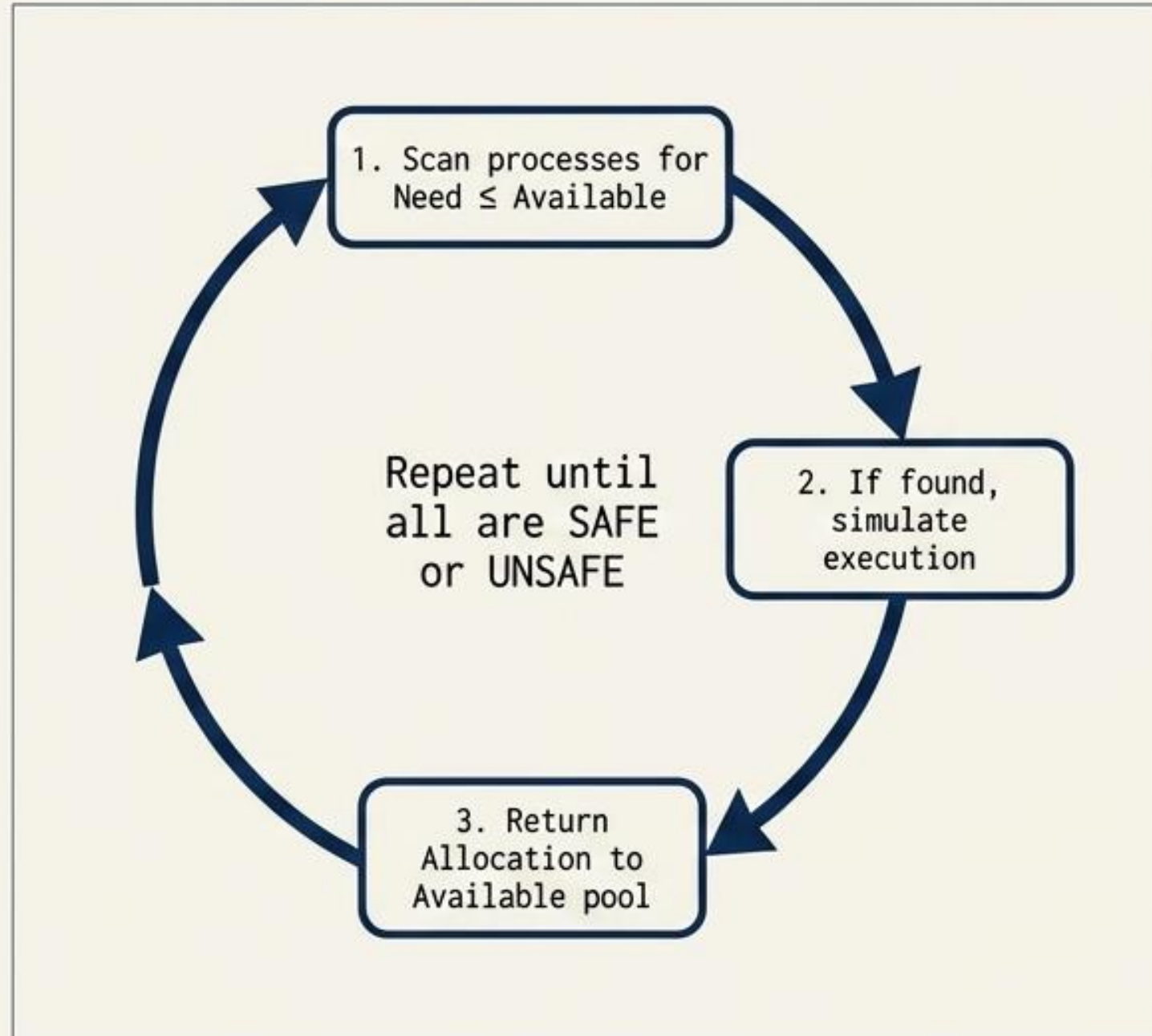
Available: A vector defining the total unallocated instances of each resource currently free in the system.

$$\text{Need} = \text{Maximum} - \text{Allocation}$$

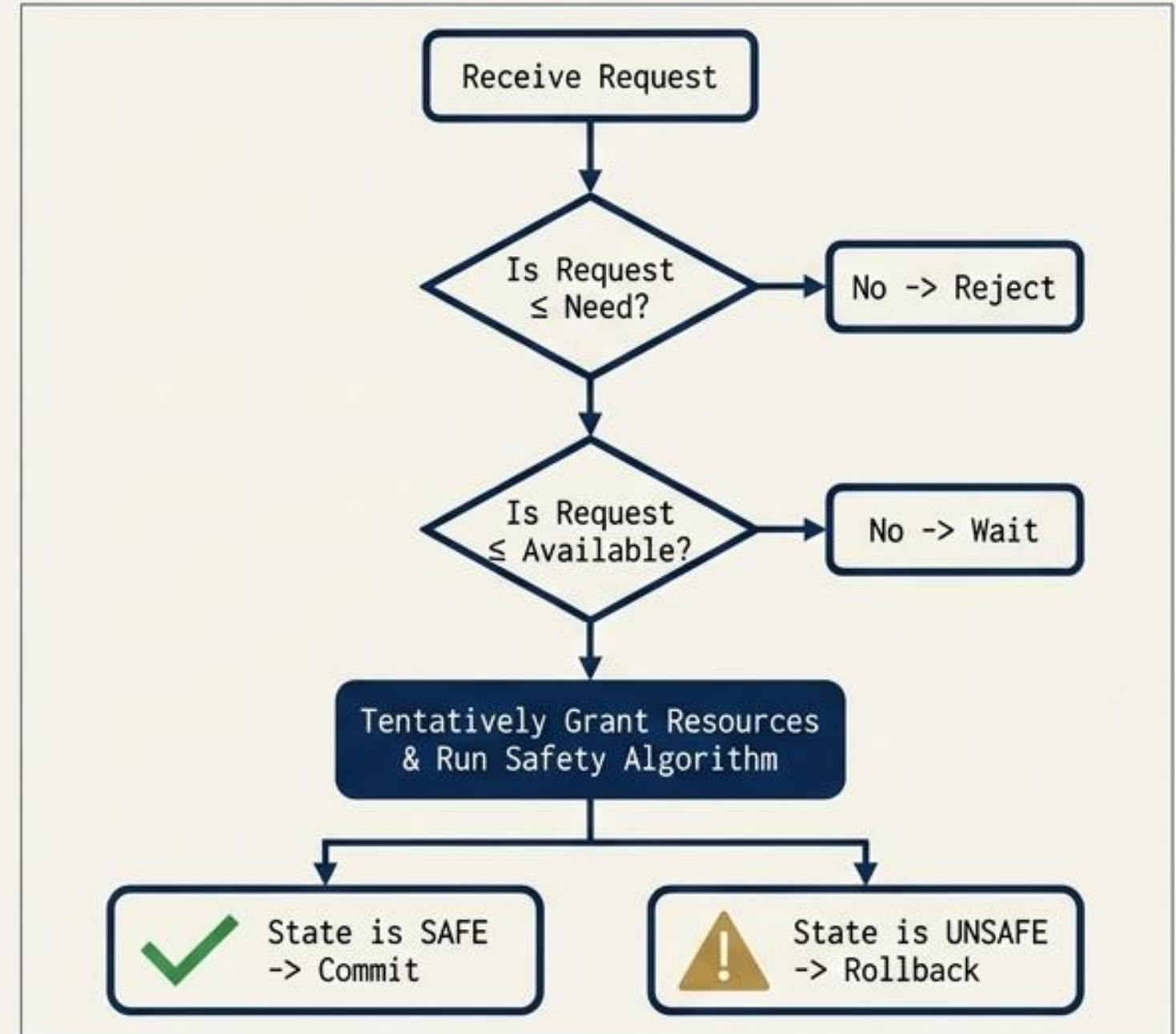


The Dual-Logic Engine: Safety and Requests

The Safety Algorithm (State Validation)



The Request Algorithm (Dynamic Allocation)



Tracing the Algorithm: Step 1 Validation

Initial Available State

[3, 3, 2]

Process	Max	Allocation	Need
P0	[7, 5, 3]	[0, 1, 0]	[7, 4, 3]
P1	[3, 2, 2]	[2, 0, 0]	[1, 2, 2]
P2	[9, 0, 2]	[3, 0, 2]	[6, 0, 0]

Scan P0: Need [7, 4, 3] > Available [3, 3, 2] → Skip

Scan P1: Need [1, 2, 2] ≤ Available [3, 3, 2] → Execute!

Action: P1 completes execution. It releases its Allocation [2, 0, 0] back to the Available pool.

Tracing the Algorithm: Halting on Unsafe State

New Available = [3, 3, 2] (Initial)
+ [2, 0, 0] (From P1)
= [5, 3, 2]

Process	Need	vs	Available
P2	[6, 0, 0]		[5, 3, 2]
P0	[7, 4, 3]		[5, 3, 2]

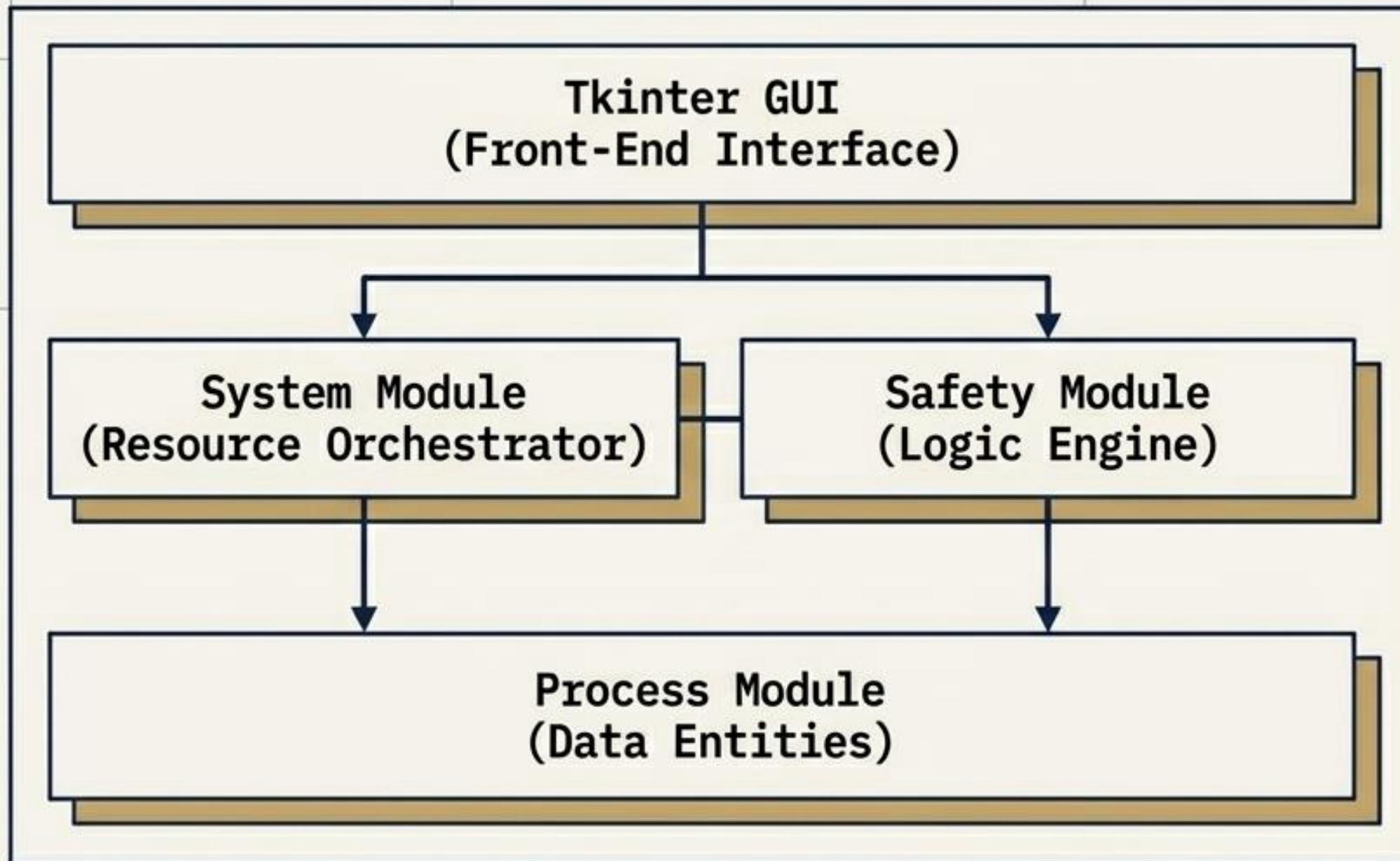
The Verdict

No remaining process can safely complete its execution with the currently available resources.

The Outcome

SYSTEM STATE: UNSAFE. The algorithm halts. If triggered dynamically, the rollback mechanism instantly denies the request, successfully avoiding deadlock.

Software Architecture and Implementation



Key Design Decisions

Computed Property

The Need matrix is computed on-the-fly (Max - Allocation) to prevent data synchronization bugs.

Decoupled Architecture

Strict separation of concerns between the underlying Python logical backend and the front-end UI.

Rollback Safety

Requests revert automatically if they trigger an unsafe path.

Simulator Setup & Configuration

System Parameters

Banker's Algorithm Simulator

Number of Processes:

Number of Resource Types:

[Configure System →](#)

Defining the structural boundaries and resource dimensions of the OS environment.

Resource Matrix

Banker's Algorithm Simulator

Step 2: Enter System Data

Available Resources

R0 R1 R2

Maximum Demand Matrix

	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Current Allocation Matrix

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

[← Back](#) [Initialize System →](#)

Initializing the Maximum potential demands and the current Allocation matrices for all system processes.

Real-Time State Visualization

Banker's Algorithm Simulator

SAFE STATE

Available Resources: 3 3 2

Safe Sequence: P1 → P3 → P0 → P2 → P4

PID	Allocation	Max Demand	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Submit a Resource Request

Process:

P0

Request:

R0 0

R1 0

R2 0

Submit Request

Safety Algorithm Walkthrough

Pass	Process	Allocation	Max	Need	Work (Available)	Fits?	Work After
2	P0	0 1 0	7 5 3	7 4 3	5 3 2	X No	5 3 2
2	P2	3 0 2	9 0 2	6 0 0	5 3 2	X No	5 3 2
2	P3	2 1 1	2 2 2	0 1 1	5 3 2	✓ Yes	7 4 3
3	P0	0 1 0	7 5 3	7 4 3	7 4 3	✓ Yes	7 5 3
4	P2	3 0 2	9 0 2	6 0 0	7 5 3	✓ Yes	10 5 5
5	P4	0 0 2	4 3 3	4 3 1	10 5 5	✓ Yes	10 5 7
✓ SAFE							Sequence: P1 → P3 → P0

Next Step

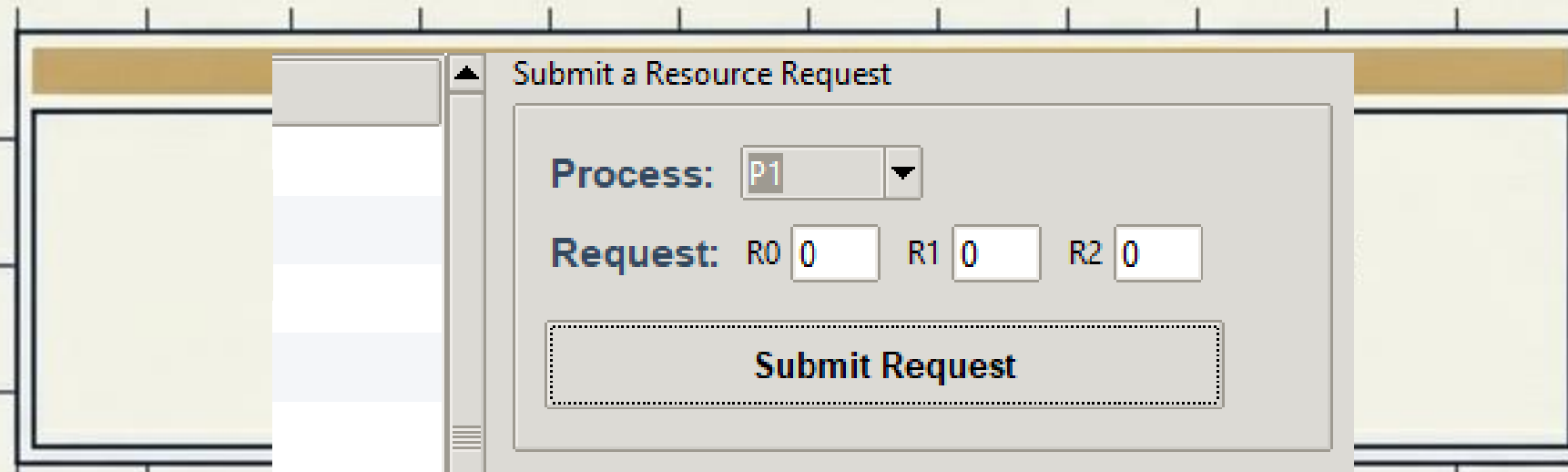
Show All

Replay

Step 8 / 8

The dynamic system dashboard. Note that the Need matrix is instantly computed and displayed alongside real-time Available resources.

Dynamic Request Handling

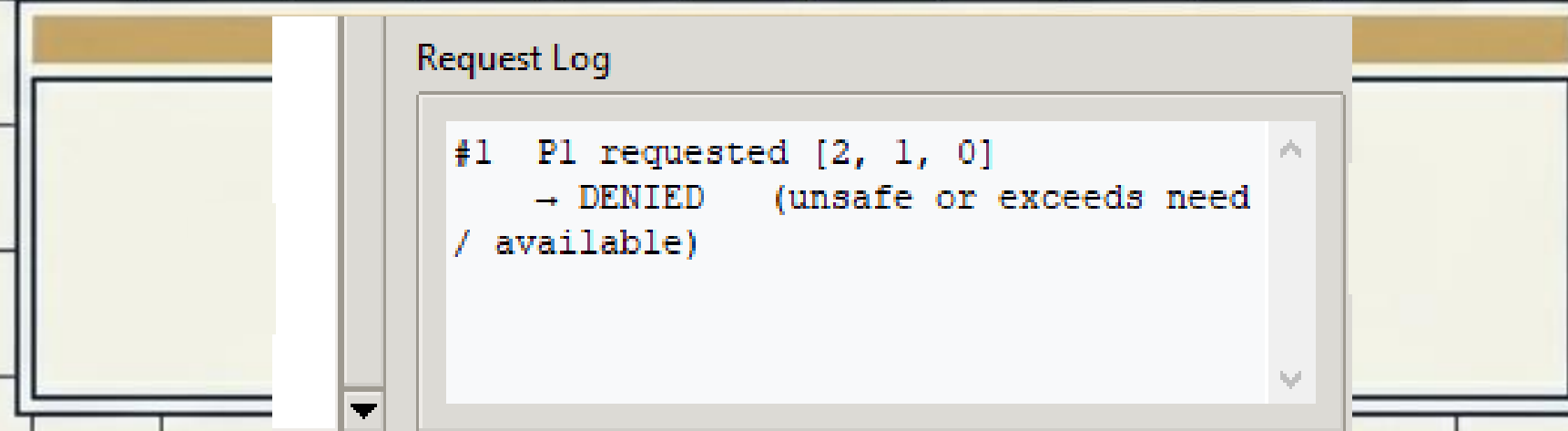


Submit a Resource Request

Process:

Request: R0 R1 R2

Simulating a live resource request initiated by a specific process.



Request Log

```
#1 P1 requested [2, 1, 0]
  → DENIED (unsafe or exceeds need
/ available)
```

The algorithm delivers its verdict, triggering either a tentative system allocation or an immediate rollback to protect the system.

Key Findings and Operational Limitations

The Findings

The Tkinter simulator successfully proves that proactive mathematical state-checking effectively neutralizes deadlocks before they manifest.



The Limitations (Theoretical vs. Practical)



1. Fixed Architecture

The algorithm assumes a rigid, unchanging number of total processes and total resource instances.



2. Clairvoyance Requirement

It requires processes to declare their Maximum resource needs entirely upfront—an unpredictable and unrealistic demand in modern computing.

Conclusion: While impractical for highly dynamic, general-purpose operating systems, Banker's Algorithm remains an exceptionally powerful conceptual model for teaching critical resource management.

Bridging OS Theory and Practice

Building this **Python-Tkinter simulator** successfully bridged the gap between **abstract Operating Systems theory** and **functional software engineering**. By **visualizing the hidden mathematics** of deadlock avoidance, we transformed a passive textbook concept into an **interactive, verifiable system**.